Instituto Tecnológico de Costa Rica

EL5859 Computación Heterogénea: II Semestre 2026

Profesor: Luis G. León-Vega, Ph.D

Evaluación: Laboratorio Semana 13

Descripción

En este laboratorio se implementará la función softmax con punto flotante de 16 y 32 bits. La primera implementación se realizará con HLS para una FPGA KV260; después se implementará el mismo algoritmo en la GPU integrada de una NVIDIA Jetson Nano y en su CPU. Finalmente se contrastarán tiempo de ejecución y consumo de energía para vectores de tamaño $N \in \{10, 100, 500, 1000\}$.

El experimento debe ejecutarse en hardware real. Las tablas son plantillas y no se deben llenar con valores estimados.

1 Objetivos

- Implementar una función con reducciones y exponencial en HLS.
- Comparar formatos flotantes de 16 bits (FP16, tipo half) y 32 bits (FP32, tipo float).
- Separar tiempo de cómputo, transferencias y lanzamiento de kernel.
- Medir energía y analizar el compromiso entre precisión, latencia, potencia y energía.

2 Fundamento

Use la versión numéricamente estable:

$$m = \max_i(x_i), \qquad y_i = \frac{\exp(x_i - m)}{\sum_{j=0}^{N-1} \exp(x_j - m)}.$$

La entrada de prueba debe estar en $[-4, 4]$ y la referencia de exactitud será una versión en double con std::exp. No se debe confundir FP16 con punto fijo: FP16 tiene signo, exponente y mantisa.

3 Preparación y protocolo

Se necesitan una KV260 con Vitis HLS/Vivado, una Jetson Nano con JetPack y CUDA. Indique si la lectura corresponde al acelerador, a la placa completa o al sistema completo.

Para todas las plataformas use los mismos vectores y semilla, realice una corrida de calentamiento, ejecute al menos cinco mediciones por configuración, reporte mediana/mínimo/máximo y conserve los datos crudos en CSV. Repita suficientes llamadas para que el instrumento de energía tenga resolución adecuada.

4 Ejercicio A: CPU de la Jetson Nano

Escriba `softmax_cpu.cpp` con variantes FP16 y FP32. Para FP16 puede usar `_Float16` o `__fp16`; si la biblioteca no ofrece exponencial FP16, convierta a FP32 para exp y para la reducción, documentando la decisión.

```
g++ -O3 -std=c++17 -march=native softmax_cpu.cpp -o softmax_cpu -lm
./softmax_cpu --precision fp16 --n 100 --reps 10000
./softmax_cpu --precision fp32 --n 100 --reps 10000
```

Mida la función y, en una segunda prueba, el programa completo. Verifique que la suma de la salida sea aproximadamente 1, que todos los resultados estén en [0,1] y que la posición del máximo coincida con la referencia. Reporte error absoluto y relativo contra FP64. Registre modo de potencia, frecuencia, temperatura y núcleos activos.

5 Ejercicio B: GPU CUDA en la Jetson Nano

Escriba `softmax_cuda.cu` usando `__half` para FP16 y float para FP32. Implemente las etapas de máximo, exponencial/reducción de suma y normalización mediante una reducción por bloque o una biblioteca de primitivas. Describa bloques, hilos y memoria.

```
nvcc -O3 -std=c++14 softmax_cuda.cu -o softmax_cuda
sudo nvpmodel -q
./softmax_cuda --precision fp16 --n 100 --reps 10000 --measure kernel
./softmax_cuda --precision fp16 --n 100 --reps 10000 --measure end-to-end
```

Use el perfilador de CUDA para medir los tiempos de ejecución. Mida también end-to-end, que incluye transferencias, lanzamientos y sincronización usando perfilado instrumentado. Registre potencia (usando tegrastats) y temperatura con tegrastats o el instrumento seleccionado.

6 Ejercicio C: HLS para KV260

Cree un proyecto de Vitis HLS para el dispositivo de la KV260 y use:

```
void softmax_hls(const data_t x[1000], data_t y[1000], int n);
```

como función superior. Defina `data_t` como FP32 y después FP16. Use float o el tipo flotante de 16 bits soportado por la versión instalada (half). No use std::exp en el kernel sintetizable: use hls::exp.

La función debe realizar la reducción del máximo, $\exp(x_i - m)$, la reducción de la suma y la normalización. Comience con una versión secuencial y aplique incrementalmente:

```
#pragma HLS PIPELINE II=1
#pragma HLS ARRAY_PARTITION variable=x cyclic factor=...
#pragma HLS DATAFLOW
```

Después de cada variante registre latencia, II, frecuencia estimada, LUT, FF, BRAM, URAM y DSP. Ejecute síntesis C y cosimulación con N=10,100,500,1000. En la KV260 reporte por separado latencia del IP, transferencias y tiempo total observado desde el host.

7 Ejercicio D: medición de energía

Mida la energía del intervalo que contiene las repeticiones:

$$E_{\text{ejecucion}} = \frac{E_{\text{fin}} - E_{\text{inicio}}}{\text{repeticiones}}, \qquad E_{\text{elemento}} = \frac{E_{\text{ejecucion}}}{N}.$$

Describa instrumento, frecuencia de muestreo, punto de medición y si se incluye el host. En la Nano fije el mismo modo de potencia para CPU y GPU. En KV260 mantenga constantes frecuencia, bitstream, temperatura y periféricos, excluyendo el warm up. Si una ejecución individual es demasiado pequeña, use un lote y divida por el número de llamadas.

8 Resultados

Complete una tabla para cada precisión. Para GPU incluya ambos tiempos; para KV260 incluya el tiempo total del host.

Plataforma	N	Tiempo (us)	Potencia (W)	Energía (uJ)	J/elem.	Error máx.
CPU	10					
CPU	100					
CPU	500					
CPU	1000					
GPU	10					
GPU	100					
GPU	500					
GPU	1000					
KV260	10					
KV260	100					
KV260	500					
KV260	1000					

Repita la tabla para FP16 y FP32. Grafique tiempo contra N, energía por llamada, energía por elemento y error contra FP64, incluyendo dispersión.

9 Preguntas

1. ¿Qué etapa domina el tiempo en cada plataforma y cómo cambia con N?

2. ¿Qué diferencia de error aparece entre FP16 y FP32? ¿Se debe al formato, la exponencial o la reducción?
3. ¿Por qué la GPU puede ser peor que la CPU para vectores pequeños?
4. ¿Qué diferencia hay entre latencia HLS y tiempo total observado desde el host?
5. ¿Qué optimización HLS mejoró más la latencia y cuánto aumentó el uso de recursos?
6. ¿Cuál plataforma minimiza tiempo y cuál energía por elemento? ¿Cambia la respuesta entre FP16 y FP32?

9.1 Entregable

Debe entregar un repositorio de Git con:

- El código separado en carpetas: CPU, GPU y FPGA.
- Un archivo `README.md` con los comandos utilizados.
- Salidas relevantes del perfilador
- Una tabla comparativa de resultados.
- Respuesta a las preguntas

9.2 Rúbrica

La calificación total del laboratorio es de 100 puntos, distribuidos de la siguiente forma:

Criterio	**Descripción**	**Puntaje**
Seguimiento de instrucciones	Entrega todos los archivos solicitados, ejecuta las herramientas indicadas, incluye los comandos usados y respeta la estructura del laboratorio.	15 pts
Calidad del reporte	Presenta resultados completos, tablas claras, evidencia suficiente de las herramientas utilizadas y datos reproducibles para cada experimento.	20 pts
Análisis del reporte	Interpreta correctamente los resultados, compara las herramientas de *profiling*, identifica *hotspots* y explica las diferencias entre mediciones.	25 pts
Propuesta de optimización y su justificación	Propone una optimización coherente con la evidencia, explica la hipótesis, compara antes y después, y concluye si el cambio mejoró el rendimiento.	25 pts
Presentación del reporte	El documento está ordenado, bien redactado, sin errores importantes de formato, con figuras o tablas legibles y conclusiones claras.	15 pts
Total		**100 pts**

Para cada criterio se puede asignar puntaje parcial según el nivel de cumplimiento: completo, mayormente completo, incompleto o ausente.

10 Referencias

- AMD, Vitis High-Level Synthesis User Guide (UG1399).
- NVIDIA, CUDA C++ Programming Guide.
- NVIDIA, Jetson Linux Developer Guide, tegrastats y modos de potencia.